```
    location ~ /\.ht {
        deny all;
    }
}
```

17.8.4.6 Step 6: Set Environment and Permissions

```
cp .env.example .env

# Update .env for MySQL connection
nano .env
# Set:
# DB_HOST=host.docker.internal
# DB_PORT=3306
# DB_DATABASE=laravel12_db
# DB_USERNAME=laraveluser
# DB_PASSWORD=securepass

# Ensure correct permissions
sudo chmod -R 755 storage bootstrap/cache
```

17.8.4.7 Step 7: Build and Run Containers

```
docker compose up -d --build
```

Access your app via: http://your-server-ip

17.8.4.8 Step 8: Run Laravel Setup Commands Inside the Container

```
docker exec -it laravel12-app bash

# Inside container shell
php artisan key:generate
php artisan config:cache
php artisan migrate --force
exit
```

17.8.5 Summary

In this lab, you've successfully deployed a **Laravel 12 application** in a **Dockerized production environment** on Ubuntu Server, while using a **MySQL database running on the host machine**. You've built containers for PHP-FPM and Nginx, configured Laravel, and accessed the app through your server IP.

This setup provides flexibility, isolation, and a solid foundation for scalable Laravel deployments in real-world environments.